Operating Systems and System Programming

Practical - 1

Title: Execution of Linux Commands Using System Calls (fork(), execvp(), wait())

Aim: To develop a C program that accepts a Linux command from the user, creates a child process using fork(), executes the command using execvp(), waits for the child process to complete using wait(), and displays the Process IDs (PIDs) of both the parent and child processes.

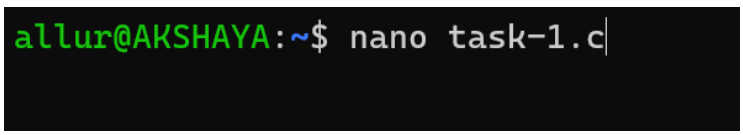
Algorithm:

1. Start the program.
2. Read a Linux command from the user.
3. Split the command into the command and its arguments.
4. Call fork() to create a child process.
5. If it is the child process:
    - Display the Child Process ID (PID).
    - Execute the command using execvp().
6. If it is the parent process:
    - Display the Parent Process ID (PID).
    - Wait for the child process to finish using wait().
7. Print the completion message.
8. End the program.

Process:

Step-1: nano task-1.c

Create a file



Step-2: Write the code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>
```

```
#include <sys/wait.h>
#include <string.h>
int main() {
    char command[100];
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    // Remove newline character
    command[strcspn(command, "\n")] = '\0';
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        execlp(command, command, NULL);
        // Executes only if execlp fails
        perror("Execution failed");
        exit(1);
    }
    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        wait(NULL);
        printf("\nChild process completed.\n");
```

```
    }
    return 0;
}
```

Step-3: press ctrl+o

Save the contents of the file.

Step-4: Press Enter

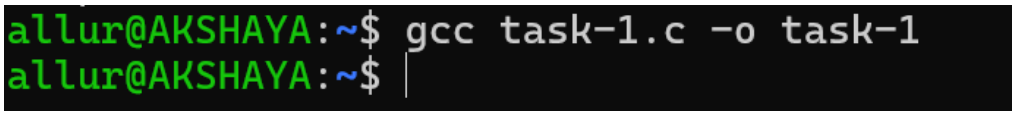
It confirms the filename and completes the save.

Step-5: Press ctrl+x

Exists the nano editor.

Step-6: gcc task-1.c -o task-1

Compiles the c source code into an executable program named task-1



Step-7: ./task-1

Runs the compiled executable from the current directory

Step-8: Enter command: date

The date command in an O.S displays the current system date and time.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: date

Parent Process
Parent PID : 787
Child PID  : 788

Child Process
Child PID : 788
Parent PID: 787
Sat Aug  1 09:34:57 UTC 2026

Child process completed.
allur@AKSHAYA:~$
```

Step-9: Enter command: uname

The uname command displays information about the operating system and system kernel, such as its name, version, and architecture.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: uname

Parent Process
Parent PID : 805
Child PID  : 806

Child Process
Child PID : 806
Parent PID: 805
Linux

Child process completed.
allur@AKSHAYA:~$
```

Step-10: Enter command:lscpu

The lscpu command displays detailed information about the system's CPU architecture, processors, cores, and threads.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: lscpu

Parent Process
Parent PID : 834
Child PID  : 835

Child Process
Child PID : 835
Parent PID: 834
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             46 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      18
  On-line CPU(s) list:       0-17
Vendor ID:                   GenuineIntel
  Model name:                Intel(R) Core(TM) Ultra 5 125H
    CPU family:              6
    Model:                   170
    Thread(s) per core:      2
    Core(s) per socket:      9
    Socket(s):               1
    Stepping:                4
    BogoMIPS:                5990.39
    Flags:                   fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clf
                             lush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_
                             good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmu
                             lqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline
                             _timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd
                             ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjus
                             t bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni x
                             saveopt xsavec xgetbv1 xsaves avx_vnni vnmi umip waitpkg gfni vaes vpclmulq
                             dq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capab
                             ilities
Virtualization features:
  Virtualization:            VT-x
  Hypervisor vendor:         Microsoft
  Virtualization type:       full
Caches (sum of all):
  L1d:                       432 KiB (9 instances)
  L1i:                       576 KiB (9 instances)
  L2:                        18 MiB (9 instances)
  L3:                        18 MiB (1 instance)
NUMA:
```

```
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-17
Vulnerabilities:
  Gather data sampling:      Not affected
  Ghostwrite:                Not affected
  Indirect target selection: Not affected
  Itlb multihit:             Not affected
  L1tf:                      Not affected
  Mds:                       Not affected
  Meltdown:                  Not affected
  Mmio stale data:           Not affected
  Old microcode:             Not affected
  Reg file data sampling:    Not affected
  Retbleed:                  Mitigation; Enhanced IBRS
  Spec rstack overflow:      Not affected
  Spec store bypass:         Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:                Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:                Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSB-eIBRS SW seq
                             uence; BHI BHI_DIS_S
  Srbds:                     Not affected
  Tsa:                       Not affected
  Tsx async abort:           Not affected
  Vmscape:                   Not affected

Child process completed.
allur@AKSHAYA:~$
```

Step-11: Enter command:lsblk

The lsblk command displays information about all available block storage devices, such as hard drives, SSDs, and their partitions.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: lsblk

Parent Process
Parent PID : 882
Child PID  : 883

Child Process
Child PID : 883
Parent PID: 882
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 356.9M  1 disk
sdb    8:16   0 159.4M  1 disk
sdc    8:32   0     2G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /mnt/wslg/distro
                               /

Child process completed.
allur@AKSHAYA:~$
```

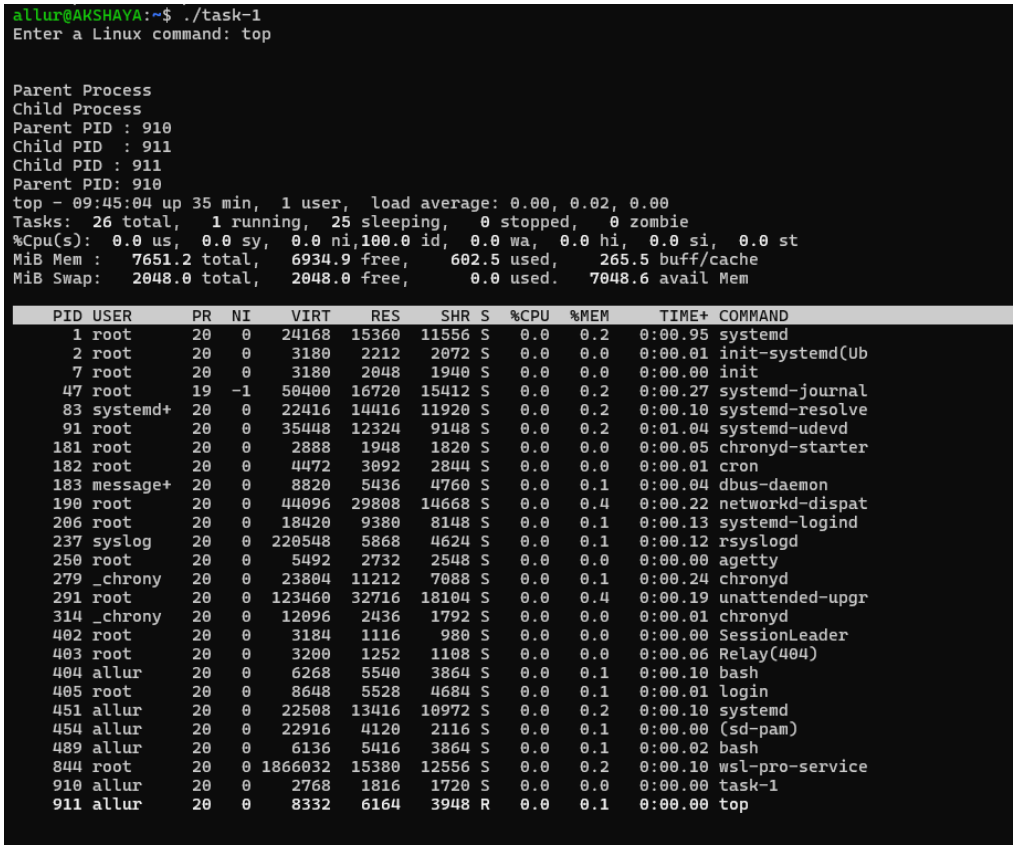
Step-12: Enter command:ps

The ps command displays information about the currently running processes on the system.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: ps

Parent Process
Parent PID : 897
Child PID  : 898

Child Process
Child PID : 898
Parent PID: 897
    PID TTY          TIME CMD
    404 pts/0    00:00:00 bash
    897 pts/0    00:00:00 task-1
    898 pts/0    00:00:00 ps

Child process completed.
allur@AKSHAYA:~$
```

Step-13: Enter command:top

The top command displays a real-time view of running processes and system resource usage, such as CPU and memory.



Conclusion:

Linux abstracts hardware resources by providing a simple interface between applications and physical hardware. It efficiently manages CPU scheduling, memory allocation, storage devices, and I/O operations through system services, allowing multiple programs to run safely and efficiently without directly interacting with the hardware.